

Versioning

Github & DVC(Data Version Control)

Agenda

- Why use Version (Source) Control Systems
- What are Git and GitHub
- Basic Git Commands
- Fundamentals of GitHub
- Using GitHub in Project Implementation
- Why DVC
- DVC and GitHub
- Building DVC pipeline

Why Version Control?

Scenario 1:

- Your program is working
- You change “just one thing”
- Your program breaks
- You change it back
- Your program is still broken--why?

Why Version Control?

- Your program worked well enough yesterday
- You made a lot of improvements last night...
(...but you haven't gotten them to work yet)
- You need to turn in your program now

Version Control for Teams?

- You change one part of a program--it works
- Your co-worker changes another part--it works
- You put them together--it doesn't work
- Some change in one part must have broken something in the other part.
- How can you merge these changes?

Version control systems

- A version control system (often called a source code control system) does these things:
- Keeps multiple (older and newer) versions of everything (source code, models with their versions, model artifacts)
- Requests comments regarding every change
- Allows “check in” and “check out” of files so you know which files someone else is working on
- Displays differences between versions

What are Git and GitHub?

- Git is a free and open-source distributed **version control system** designed to handle everything from small to very large projects with speed and efficiency.
- GitHub is a web-based Git repository **hosting service**, which offers all of the distributed revision control and source code management (SCM) functionality of Git as well as adding its own features.

How to setup Git and GitHub

Option 1: Download and install the latest version of [Desktop GitHub](#). This will automatically install Git and keep it up to-date for you.

Option 2: Download [Git](#) and use Git Bash to access GitHub via Git commands

Basic Git Commands

Github

Initialize, Commit and Push

- On your computer, open the Git Bash/CMD application
- Create an Empty Github repository and copy its http link.
- Set up identity:

```
git config --global user.name "Your Name"
```

```
git config --global user.email "youremail@example.com"
```
- Initialize your project:

```
mkdir house-price-mlops
```

```
cd house-price-mlops
```

```
git init
```

init and .git repository

When you said `git init` in your project directory, or when you cloned an existing project, you created a repository

- The repository is a subdirectory named `.git` containing various files
- The dot indicates a “hidden” directory
- You do not work directly with the contents of that directory;
various git commands do that for you

Github

- git branch
- git remote -v
- git branch -M main
- git remote add origin <GITHUB-URL> # add remote github link
- git checkout -b main # ensure branch is main
- git add flask # stage the folder/file or . current
- git commit -m "First Commit 01"
- # A message telling what you have done is required
- git push -u origin main # push staged content to remote repo

Initializes Git, creates main, links to remote, commits, and pushes the folder.

Initialize, Commit and Push

Commits and Graphs

- A commit is when you tell git that a change (or addition) you have made is
- ready to be included in the project
- When you commit your change to git, it creates a commit object
 - A commit object represents the complete state of the project, including all the files in the project
 - The very first commit object has no “parents”
 - Usually, you take some commit object, make some changes, and create a new commit object; the original commit object is the parent of the new commit object
- You can also merge two commit objects to form a new one
- Commit objects forms a directed graph
- Git is all about using and manipulating this graph

Github

Pull , Clone

Downloads new changes from the remote repo and merges them into your existing local repo.

- `git pull origin main`

Copies an entire remote repository to your local machine for the first time.

- `git clone https://github.com/yourname/your-repo.git`

Initializes Git, creates main, links to remote, commits, and pushes the folder.

Github

Clone, Branch, Push

- git remote remove origin (if already remote is set)
- git clone <https://github.com/yourname/your-repo.git> # Clone
- cd your-repo

Create & switch to new branch name feature/userB-change

- git checkout -b feature/userB-change

After making changes in code.

- git add .
- git commit -m "UserB: update homepage UI"
- git push -u origin feature/userB-change # Push staged Contents

Github

Branch & Merge

- **Branch** is an independent line of development. **main** is the primary branch in each GitHub repository and **merge** bring changes from one branch into another.

```
git branch    # shows current branch (default: main or master)
```

```
git checkout main # select main as current branch
```

```
# create & switch to feature branch
```

```
git checkout -b feature/add-login
```

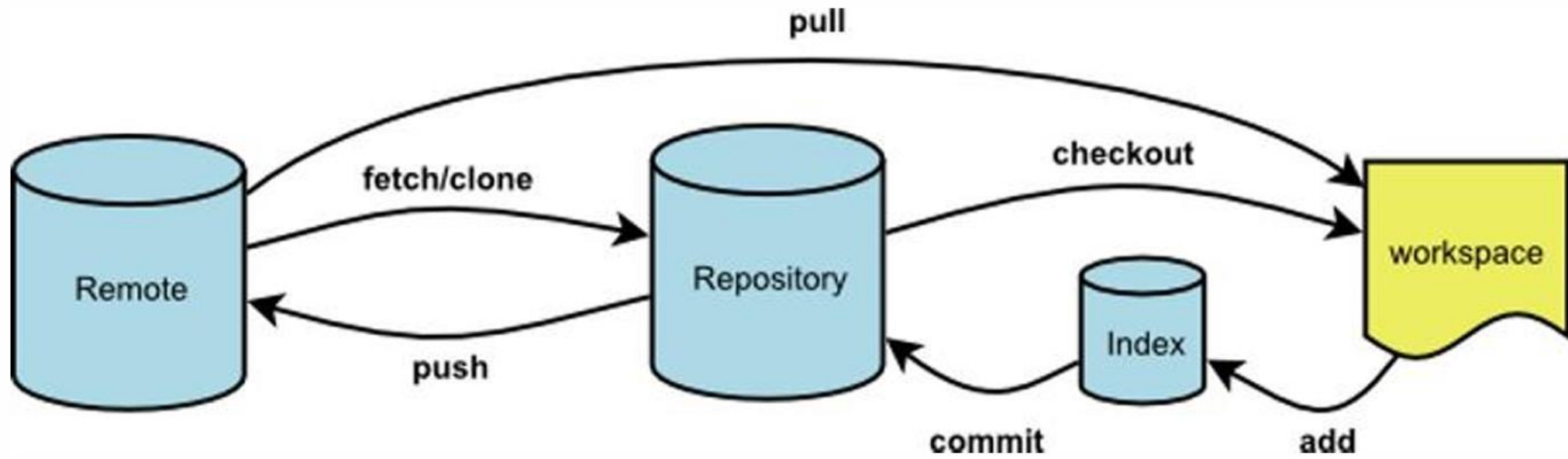
```
git add .
```

```
git commit -m "feat: add login page"
```

```
git push -u origin feature/add-login
```

```
git merge main    # Merges feature/add-login to main branch
```

Git & GitHub



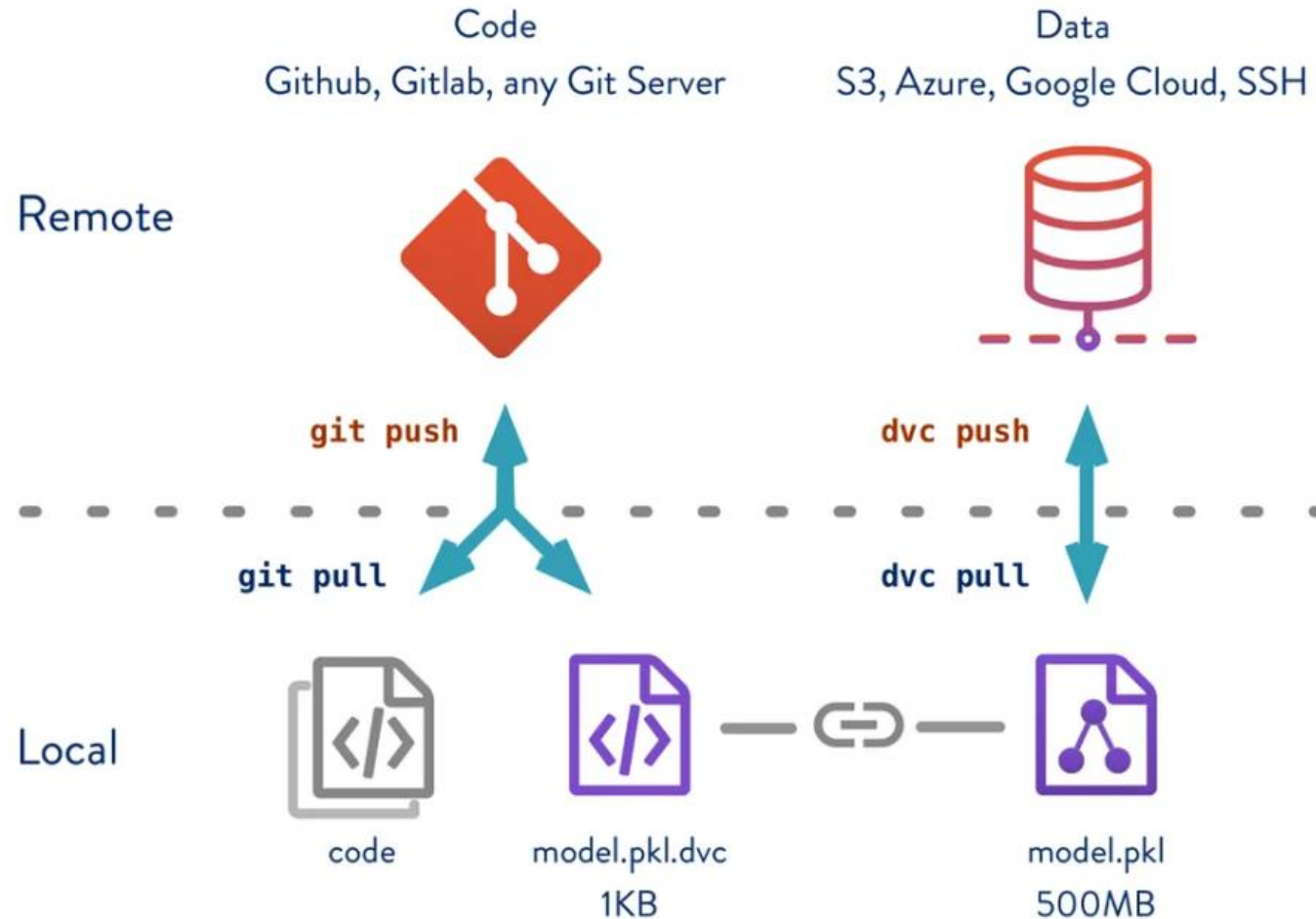
Data Version Control (DVC)

Learn collaboration between DVC and GitHub.

DVC

- DVC, or Data Version Control, is an open-source tool specifically designed for data science and machine learning projects.
- It works to version larger datasets and models(their artifacts too) allowing users to track data changes, collaborate efficiently.
- Git keeps track of changes you make to your code, DVC keeps track of changes you make to your data.
- Git of Data

Analogy of Git and DVC



DVC and Git

1. `pip install dvc`
 2. `dvc init` # Sets up DVC to work alongside Git
 3. `dvc add data/` # Starts the tracking of our data
- **It creates a `data.dvc` file which contains data unique hash ID to track the data folder and in terminal**
 - **Add this `data.dvc` file in git.**

```
! data.dvc
1  outs:
2  - md5: 29d0520fa8aacebd00816e93c856ba8f.dir
3    size: 14
4    nfiles: 5
5    hash: md5
6    path: data
7
```

DVC and Git

- 3. `git add data.dvc` # Commit changes to Git
- 4. `dvc status` # check the status of data, if data changes
- 5. `dvc commit` # commit this change to sync data.dvc file

Now Commit the DVC tracking file to GitHub

- 6. `git add data.dvc`
- 7. `git commit -m "Track dataset using DVC"`

Configure Remote Storage for DVC

- Google Drive:

```
dvc remote add -d gdrive gdrive://<your-drive-folder-id>
```

```
dvc push
```

This uploads large data files to Google Drive, but only lightweight .dvc files are stored in GitHub.

DVC Pipeline

- YAML is a human-readable configuration format — like JSON.
- It's often used to store:
 - Model hyperparameters
 - Data paths
 - Model configuration options

```
train:
  learning_rate: 0.001
  batch_size: 64
  epochs: 20

model:
  type: resnet50
  dropout: 0.3

data:
  train_path: data/train.csv
  test_path: data/test.csv
```

Params.yaml

DVC Pipeline

- YAML is a human-readable configuration format — like JSON.
- It's often used to store:
 - Model hyperparameters
 - Data paths
 - Model configuration options

```
stages:  
  train:  
    cmd: python src/train.py  
    deps:  
      - src/train.py  
      - data/raw/house_data.csv  
      - params.yaml  
    outs:  
      - models/house_price_model.pkl
```

dvc.yaml

DVC Pipeline

- YAML is a human-readable configuration format — like JSON.
- It's often used to store:
 - Model hyperparameters
 - Data paths
 - Model configuration options

Run the pipeline:

dvc repro

```
stages:
  train:
    cmd: python src/train.py
    deps:
      - src/train.py
      - data/raw/house_data.csv
      - params.yaml
    outs:
      - models/house_price_model.pkl
```

dvc.yaml

Push Model and Data

Push your model and dataset to DVC remote:

```
dvc push
```

Push your Git commits:

```
git add .
```

```
git commit -m "Add training pipeline with DVC"
```

```
git push origin main
```

Versioning and Collaboration

- To clone repository:

```
git clone https://github.com/<your-username>/house-price-mlops.git
```

```
cd house-price-mlops
```

```
pip install -r requirements.txt
```

```
dvc pull # downloads data & models from DVC remote
```

They get exact same version of **code + data + models**.